

Apostila

Persistência de Dados em Java com JDBC

Guia prático para alunos

Professora: Simone Lacerda

Java

JDBC

MySQL

VSCode



Roteiro da jornada

Fundamentos, conexão, projeto simples e CRUD completo.

O que esta apostila apresenta

O material introduz o conceito de persistência de dados, explica o papel do JDBC e conduz a um projeto Java conectado ao MySQL. O desenvolvimento passa por um teste simples e avança até a organização de um CRUD com Produto, Conexao, ProdutoDAO e Main.

Persistência Salvar dados de forma permanente.	JDBC Comunicar Java com bancos relacionais.
MySQL Banco utilizado para armazenar produtos.	CRUD Criar, ler, atualizar e deletar registros.

Sumário interativo

- 01 Fundamentos e objetivos
- 02 Requisitos do projeto
- 03 Criando o banco MySQL
- 04 Projeto Java simples
- 05 Compilar e executar
- 06 Criando o projeto no VSCode
- 07 Projeto via linha de comando
- 08 Estrutura do projeto CRUD
- 09 Produto.java
- 10 Conexao.java
- 11 ProdutoDAO.java
- 12 Main.java
- 13 Conclusão, referências e checklist

Fundamentos e objetivos

Entenda por que os dados precisam persistir e qual o papel do JDBC.

[< voltar ao sumário](#)

Justificativa

No mundo do desenvolvimento de sistemas, salvar informações de forma permanente (persistência de dados) é essencial. Compreender como conectar uma aplicação Java a um banco de dados é uma habilidade fundamental para quem está aprendendo programação, especialmente em cursos técnicos.

Objetivos

- Entender o que é persistência de dados e como usá-la com Java.
- Aprender o que é JDBC e como utilizá-lo para se conectar a um banco de dados MySQL.
- Desenvolver um projeto CRUD (Criar, Ler, Atualizar, Deletar) usando a linguagem Java.
- Utilizar o Visual Studio Code (VSCode) como ambiente de desenvolvimento.

1. O que é persistência de dados?

Persistência de dados significa guardar as informações de um sistema de forma que elas não se percam quando o programa for fechado. Isso normalmente é feito com o uso de bancos de dados, como o MySQL.

2. O que é JDBC?

JDBC (Java Database Connectivity) é uma API do Java que permite a comunicação entre a linguagem Java e bancos de dados relacionais, como o MySQL.

Ideia central

A persistência mantém os dados depois que o programa termina; o JDBC é o mecanismo usado pelo Java para conversar com o banco.

Requisitos do projeto

Prepare as ferramentas indicadas no tutorial antes de começar.

[< voltar ao sumário](#)

- Java JDK instalado (recomenda-se JDK 17)
- MySQL instalado
- VSCode com a extensão "Extension Pack for Java"
- Driver JDBC para MySQL (arquivo .jar)
- Terminal ou CMD

Mapa das ferramentas

Java JDK Compila e executa a aplicação Java.	VSCode Ambiente de desenvolvimento utilizado pelo tutorial.
Driver JDBC Arquivo .jar que permite a conexão Java-MySQL.	MySQL Banco de dados relacional do projeto.
Terminal/CMD Usado nas etapas de compilação e linha de comando.	CRUD Conjunto das quatro operações de manipulação de dados.

Antes de avançar

Confirme que o MySQL está acessível e que o driver JDBC foi obtido. O projeto simples da próxima etapa depende desses dois itens.

PASSO 3

Criando o banco MySQL

Crie o banco loja e a tabela produtos conforme o tutorial.

03

[< voltar ao sumário](#)

Abra o MySQL e crie um banco de dados com a tabela produtos:

```
CREATE DATABASE loja;

USE loja;

CREATE TABLE produtos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100),
  preco DECIMAL(10,2)
);
```

Script SQL do tutorial para criar o banco loja e a tabela produtos.

Estrutura definida no banco

Campo	Definição no tutorial	Papel
id	INT AUTO_INCREMENT PRIMARY KEY	Identificador do produto.
nome	VARCHAR(100)	Nome do produto.
preco	DECIMAL(10,2)	Preço do produto.

Checkpoint

Banco loja criado e tabela produtos pronta antes de iniciar o teste Java.

Projeto Java simples

Faça primeiro um teste de conexão e listagem em um único Main.java.

[< voltar ao sumário](#)

Antes de criarmos o projeto completo com múltiplas classes, o tutorial propõe um teste simples para garantir que conseguimos nos conectar ao banco de dados MySQL e listar os produtos da tabela.

Passo a passo

- Certifique-se de que o banco de dados loja e a tabela produtos já estão criados.
- Coloque o driver mysql-connector-j-8.x.x.jar na pasta lib/.
- Crie o arquivo Main.java com o conteúdo apresentado no tutorial.

```
1
2 import java.sql.*;
3
4 public class Main {
5     @SuppressWarnings("ConvertToTryWithResources")
6     public static void main(String[] args) {
7         // Dados de conexão
8         String url = "jdbc:mysql://localhost:3306/loja";
9         String usuario = "root";
10        String senha = ""; // Substitua pela sua senha do MySQL
11
12        try {
13            // Estabelece a conexão com o banco de dados
14            Connection conexao = DriverManager.getConnection(url, usuario, senha);
15            System.out.println("Conexão estabelecida com sucesso!");
16
17            // Cria e executa uma consulta SQL
18            String sql = "SELECT * FROM produtos";
19            Statement stmt = conexao.createStatement();
20            ResultSet resultado = stmt.executeQuery(sql);
21
22            // Exibe os produtos no console
23            System.out.println("Lista de Produtos:");
24            while (resultado.next()) {
25                int id = resultado.getInt("id");
26                String nome = resultado.getString("nome");
27                double preco = resultado.getDouble("preco");
28
29                System.out.println(id + " - " + nome + " - R$" + preco);
30            }
31
32            // Fecha a conexão
33            resultado.close();
34            stmt.close();
35            conexao.close();
36
37        } catch (SQLException e) {
38            System.out.println("Erro ao conectar: " + e.getMessage());
39        }
40    }
41 }
42
```

Main.java (versão simples) apresentado no material.

Entendendo o Main.java simples

Leia a função de cada objeto JDBC antes de executar.

[< voltar ao sumário](#)

- String url, usuario, senha: dados para se conectar ao banco. Troque "sua_senha" pela senha do seu MySQL.
- DriverManager.getConnection(...): estabelece a conexão com o banco.
- Statement stmt: cria o comando que será enviado ao banco de dados.
- ResultSet resultado: guarda os dados retornados pela consulta `SELECT * FROM produtos`.
- while (resultado.next()): percorre todos os registros retornados.
- System.out.println(...): exibe no console o ID, nome e preço de cada produto.
- resultado.close(), stmt.close(), conexao.close(): encerram a conexão com o banco e liberam os recursos.

Fluxo do teste

Java abre a conexão, envia um SELECT, percorre o ResultSet e mostra os produtos no console.

Compilar e executar

Use os comandos de terminal apresentados no tutorial.

[< voltar ao sumário](#)

No terminal, navegue até o diretório do projeto e execute:

```
javac -cp "lib/*" Main.java
java -cp "lib/*:." Main
```

Comandos de compilação e execução mostrados no material.

No Windows, o material orienta usar ; em vez de : no classpath:

```
java -cp "lib/*;." Main
```

Comando indicado no tutorial para o Windows.

O que observar

Compilação javac utiliza o driver disponível na pasta lib.	Execução java inicia a classe Main com o classpath configurado.
Console O resultado esperado é a conexão e a listagem dos produtos.	Driver O arquivo .jar precisa estar disponível para a aplicação.

Criando o projeto no VSCode

Opção recomendada pelo tutorial.

06

[< voltar ao sumário](#)

- No VSCode, pressione Ctrl + Shift + P e selecione Java: Create Java Project.
- Escolha No build tools.
- Nomeie o projeto como CrudJDBC.
- Dentro do projeto, crie a pasta lib e coloque o driver JDBC MySQL (mysql-connector-j-x.x.xx.jar).
- Clique com o botão direito no projeto → Properties → Java Build Path → Add JARs... → adicione o JAR do driver.
- Crie as classes Produto.java, Conexao.java, ProdutoDAO.java e Main.java.

Opção recomendada no material

Esta sequência utiliza VSCode e um projeto sem build tools, com o driver JDBC adicionado manualmente à pasta lib.

Projeto via linha de comando

Alternativa para criar a estrutura manualmente.

[< voltar ao sumário](#)

```
mkdir CrudJDBC
cd CrudJDBC
mkdir src
mkdir lib
cd src
code .
```

Comandos do tutorial para criar CrudJDBC, src e lib.

Coloque os arquivos .java dentro da pasta src, e o driver .jar dentro de lib. Para compilar:

```
javac -cp "lib/*" src/*.java
java -cp "lib/*:src" Main
```

Comandos de compilação e execução apresentados no tutorial.

Duas formas, mesmo objetivo

A opção do VSCode e a opção de linha de comando levam à mesma organização lógica: código-fonte separado do driver JDBC.

Estrutura do projeto CRUD

Visualize como os arquivos ficam organizados.

[< voltar ao sumário](#)

```
CrudJDBC/  
|  
├─ lib/  
|   └─ mysql-connector-j-8.x.x.jar  
├─ src/  
|   ├── Produto.java  
|   ├── Conexao.java  
|   ├── ProdutoDAO.java  
|   └─ Main.java
```

Estrutura CrudJDBC apresentada no tutorial.

Responsabilidade dos arquivos

Arquivo	Responsabilidade
Produto.java	Representa os dados de um produto.
Conexao.java	Centraliza a conexão com o banco MySQL.
ProdutoDAO.java	Contém as operações de acesso e manipulação dos dados.
Main.java	Programa principal e menu de interação.

Produto.java

Classe que representa os dados da tabela produtos.

[< voltar ao sumário](#)

```
1 public class Produto {  
2     private int id;  
3     private String nome;  
4     private double preco;  
5  
6     public Produto(String nome, double preco) {  
7         this.nome = nome;  
8         this.preco = preco;  
9     }  
10  
11     // Getters e Setters  
12     public int getId() { return id; }  
13     public void setId(int id) { this.id = id; }  
14     public String getNome() { return nome; }  
15     public void setNome(String nome) { this.nome = nome; }  
16     public double getPreco() { return preco; }  
17     public void setPreco(double preco) { this.preco = preco; }  
18 }
```

Código Produto.java do tutorial.

Descrição

A classe Produto representa um produto da nossa tabela no banco de dados. Ela possui três atributos:

- id: identificador único do produto, gerado automaticamente pelo MySQL.
- nome: representa o nome do produto.
- preco: representa o valor do produto.
- Também há um construtor que recebe nome e preço e métodos getters e setters para acessar e alterar os valores dos atributos.

Conexao.java

Classe responsável por criar a conexão com o MySQL.

[< voltar ao sumário](#)

```
1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.SQLException;
4
5 public class Conexao {
6     private static final String URL = "jdbc:mysql://localhost:3306/loja";
7     private static final String USER = "root";
8     private static final String PASSWORD = "sua_senha";
9
10    public static Connection conectar() throws SQLException {
11        return DriverManager.getConnection(URL, USER, PASSWORD);
12    }
13 }
```

Código Conexao.java do tutorial.

Descrição

A classe Conexao é responsável por criar a conexão com o banco de dados MySQL.

- O método conectar() retorna um objeto Connection, usado para enviar comandos SQL.
- DriverManager.getConnection(...) utiliza a URL do banco, o usuário (root) e a senha configurada.
- Essa classe ajuda a evitar repetição de código toda vez que precisamos conectar com o banco.

Atenção

No código do tutorial, substitua "sua_senha" pela senha configurada no seu MySQL.

ProdutoDAO.java

Data Access Object: acesso e manipulação da tabela produtos.

[< voltar ao sumário](#)

```
1 import java.sql.*;
2 import java.util.*;
3
4 public class ProdutoDAO {
5
6     @SuppressWarnings("CallToPrintStackTrace")
7     public void inserir(Produto p) {
8         String sql = "INSERT INTO produtos (nome, preco) VALUES (?, ?)";
9
10        try (Connection conn = Conexao.conectar();
11             PreparedStatement stmt = conn.prepareStatement(sql)) {
12
13            stmt.setString(1, p.getNome());
14            stmt.setDouble(2, p.getPreco());
15            stmt.executeUpdate();
16            System.out.println("Produto inserido com sucesso!");
17        } catch (SQLException e) {
18            e.printStackTrace();
19        }
20    }
21
22    @SuppressWarnings("CallToPrintStackTrace")
23    public List<Produto> listar() {
24        List<Produto> lista = new ArrayList<>();
25        String sql = "SELECT * FROM produtos";
26
27        try (Connection conn = Conexao.conectar();
28             Statement stmt = conn.createStatement();
29             ResultSet rs = stmt.executeQuery(sql)) {
30
31            while (rs.next()) {
32                Produto p = new Produto(rs.getString("nome"),
33                                         rs.getDouble("preco"));
34                p.setId(rs.getInt("id"));
35                lista.add(p);
36            }
37        } catch (SQLException e) {
38            e.printStackTrace();
39        }
40        return lista;
41    }
42 }
```

Primeira parte do ProdutoDAO.java: inserção e listagem.

ProdutoDAO.java - continuação

Atualização, exclusão e leitura das operações CRUD.

[< voltar ao sumário](#)

```
42
43 @SuppressWarnings("CallToPrintStackTrace")
44 public void atualizar(Produto p) {
45     String sql = "UPDATE produtos SET nome=?, preco=? WHERE id=?";
46
47     try (Connection conn = Conexao.conectar();
48         PreparedStatement stmt = conn.prepareStatement(sql)) {
49
50         stmt.setString(1, p.getNome());
51         stmt.setDouble(2, p.getPreco());
52         stmt.setInt(3, p.getId());
53         stmt.executeUpdate();
54         System.out.println("Produto atualizado!");
55     } catch (SQLException e) {
56         e.printStackTrace();
57     }
58 }
59
60 @SuppressWarnings("CallToPrintStackTrace")
61 public void deletar(int id) {
62     String sql = "DELETE FROM produtos WHERE id=?";
63
64     try (Connection conn = Conexao.conectar();
65         PreparedStatement stmt = conn.prepareStatement(sql)) {
66
67         stmt.setInt(1, id);
68         stmt.executeUpdate();
69         System.out.println("Produto deletado!");
70     } catch (SQLException e) {
71         e.printStackTrace();
72     }
73 }
74 }
```

Segunda parte do ProdutoDAO.java: atualização e exclusão.

As quatro operações do CRUD

inserir()

Adiciona um novo produto ao banco.

listar()

Retorna todos os produtos da tabela.

atualizar()

Altera nome e preço de um produto, dado o ID.

deletar()

Remove um produto do banco, usando seu ID.

try-with-resources

Todos os métodos usam try-with-resources, o que garante que as conexões sejam fechadas corretamente ao final do uso.

Main.java

Programa principal e menu de interação.

[< voltar ao sumário](#)

```
1  import java.util.*;
2
3  public class Main {
4      @SuppressWarnings("ConvertToTryWithResources")
5      public static void main(String[] args) {
6          ProdutoDAO dao = new ProdutoDAO();
7          Scanner sc = new Scanner(System.in);
8
9          System.out.println(
10             "1 - Inserir | 2 - Listar | 3 - Atualizar | 4 - Deletar"
11         );
12         int opcao = sc.nextInt();
13         sc.nextLine(); // Limpa buffer
14
15         switch (opcao) {
16             case 1:
17                 System.out.print("Nome: ");
18                 String nome = sc.nextLine();
19                 System.out.print("Preço: ");
20                 double preco = sc.nextDouble();
21                 Produto p = new Produto(nome, preco);
22                 dao.inserir(p);
23                 break;
24             case 2:
25                 for (Produto prod : dao.listar()) {
26                     System.out.println(
27                         prod.getId() + " - " +
28                         prod.getNome() + " - R$" +
29                         prod.getPreco());
30                 }
31                 break;
```

Primeira parte do Main.java: menu e operações iniciais.

Main.java - continuação

Atualizar, deletar e finalizar a execução.

[< voltar ao sumário](#)

```
32         case 3:
33             System.out.print("ID do produto: ");
34             int id = sc.nextInt();
35             sc.nextLine();
36             System.out.print("Novo nome: ");
37             nome = sc.nextLine();
38             System.out.print("Novo preço: ");
39             preco = sc.nextDouble();
40             p = new Produto(nome, preco);
41             p.setId(id);
42             dao.atualizar(p);
43             break;
44         case 4:
45             System.out.print("ID do produto a deletar: ");
46             id = sc.nextInt();
47             dao.deletar(id);
48             break;
49         default:
50             System.out.println("Opção inválida.");
51     }
52     sc.close();
53 }
54 }
55
56
```

Segunda parte do Main.java apresentada no tutorial.

Descrição

Esse é o programa principal que o usuário executa. Ele funciona como um menu no console, permitindo escolher uma das quatro ações:

- Inserir um novo produto.
- Listar todos os produtos.
- Atualizar um produto já existente.
- Deletar um produto pelo ID.
- Usa a classe Scanner para ler as entradas do teclado.
- Cria objetos da classe Produto e chama os métodos da ProdutoDAO.
- É um exemplo de interação básica com o usuário via terminal.

Conclusão e referências

Fechamento do tutorial e checklist do aluno.

[< voltar ao sumário](#)

Conclusão

Aprender a persistir dados com Java e MySQL utilizando JDBC é um passo importante para quem deseja trabalhar com desenvolvimento de software. O projeto apresentado mostra de forma prática como criar, ler, atualizar e deletar registros no banco de dados. Dominar essa conexão entre backend e banco de dados é essencial no mercado de trabalho técnico.

Checklist interativo

- ☐ Entendi o conceito de persistência de dados.
- ☐ Entendi o papel do JDBC.
- ☐ Criei o banco loja e a tabela produtos.
- ☐ Configurei o driver JDBC na pasta lib.
- ☐ Executei o Main.java simples.
- ☐ Organizei Produto, Conexao, ProdutoDAO e Main.
- ☐ Testei as quatro operações do CRUD.

Referências

[Oracle - Introdução ao JDBC](#)

[MySQL - Documentação oficial](#)

[Visual Studio Code - Extension Pack for Java](#)

[Loiane Groner - Curso de Java e Banco de Dados](#)

Material didático - ETEC Bento Quirino / Centro Paula Souza
Professora: Simone Lacerda